

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO

Se requiere una página web de presentación corporativa para una productora audiovisual. El stack está explícitamente restringido a HTML5 + CSS + JS puro (sin frameworks de backend). El sitio es estático con 4 secciones (Home, Servicios, Quiénes Somos, Contacto), diseño responsivo, formulario de contacto con validación client-side, y uso obligatorio de placeholders para imágenes. No existe base de datos. El formulario de contacto, según el requerimiento, debe 'enviar correo', lo cual en un stack puramente estático se resuelve mediante un servicio de terceros (Formsprees/EmailJS) o simulación JS con mensaje de confirmación — dado que el requerimiento excluye backend propio, se implementará con EmailJS (CDN) o simulación de envío con feedback visual, sin almacenamiento de datos.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Sitio Estático Multi-Sección (Single HTML + CSS + JS)

- Un único archivo `index.html` con navegación por anclas (#home, #servicios, #quienes-somos, #contacto) — NO 4 archivos HTML separados.
- Un archivo `styles.css` principal con variables CSS custom properties para theming.
- Un archivo `main.js` para interactividad (navbar sticky, smooth scroll, validación de formulario, animaciones de entrada, toggle de idioma ES/EN).
- Integración de EmailJS vía CDN para el envío del formulario sin backend.

3. JUSTIFICACIÓN BASADA EN YAGNI/KISS

- **¿Por qué NO microservicios ni backend propio?** El requerimiento es explícito: 'HTML5, CSS y JS solamente'. No hay lógica de negocio que requiera servidor. YAGNI aplicado.
- **¿Por qué NO 4 archivos HTML separados?** Una SPA con anclas reduce la complejidad de mantenimiento, elimina la duplicación del navbar/footer en 4 archivos, mejora el rendimiento (una sola carga) y simplifica el despliegue. El comportamiento de 'páginas separadas' se logra con scroll suave y URL hash.
- **¿Por qué NO un framework JS (React/Vue)?** El requerimiento prohíbe explícitamente frameworks. Además, para contenido estático de presentación, añadir un framework sería over-engineering puro. KISS aplicado.
- **¿Por qué NO CMS integrado ahora?** El requerimiento menciona CMS como deseable pero el stack es HTML/CSS/JS puro. La integración CMS queda fuera del alcance actual (excluido en functional_spec). Se estructura el HTML con comentarios claros para facilitar edición manual o integración futura.
- **¿Por qué EmailJS y no un backend PHP/Node?** El stack está restringido. EmailJS opera 100% desde el cliente vía CDN, cumple el requisito de 'enviar correo' sin almacenar datos en base de datos, y satisface el criterio de seguridad de no persistir datos sensibles.
- **Multiidioma ES/EN:** Se implementa con un objeto de traducciones en JS y atributos `data-i18n` en el HTML. Sin librerías externas. Solución mínima viable.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (EmailJS):** Dependencia de servicio de terceros para el formulario. Mitigación: el formulario muestra mensaje de error claro si el envío falla (Escenario 6 cubierto).
- **Riesgo 2 (Single HTML):** Si el contenido crece significativamente, el archivo puede volverse difícil de mantener. Mitigación: estructura modular con comentarios de sección bien definidos.
- **Riesgo 3 (Sin HTTPS propio):** HTTPS depende del proveedor de hosting (precondición del cliente, fuera del alcance del código).

- **Riesgo 4 (CDN para rendimiento):** La CDN es responsabilidad del hosting, no del código entregado.
- **Riesgo 5 (Protección XSS):** Se implementa sanitización básica en JS del lado cliente. La protección robusta requeriría backend (fuera de alcance).

5. CONTRATO DE INTERFACES

- Imágenes: TODAS las etiquetas `<img>` usan URLs de `https://placeholder.co/` o `https://picsum.photos/`. Cero archivos locales de imagen.
- EmailJS: Requiere configuración de `EMAILJS_SERVICE_ID`, `EMAILJS_TEMPLATE_ID`, `EMAILJS_PUBLIC_KEY` como constantes en `main.js` (documentadas con comentarios TODO).
- Anclas de navegación: `#home`, `#servicios`, `#quienes-somos`, `#contacto`.
- Clases CSS críticas: `.navbar`, `.hero`, `.section`, `.card`, `.contact-form`.
- Atributos i18n: `data-i18n=[clave]` para el sistema de traducción ES/EN.

2. MÉTRICAS DE ITERACIÓN (QA)

- productora-web/index.html | Estado: APROBADO | Iteraciones: 1
- productora-web/js/contact.js | Estado: APROBADO | Iteraciones: 1
- productora-web/js/main.js | Estado: APROBADO | Iteraciones: 1
- productora-web/css/styles.css | Estado: APROBADO | Iteraciones: 1
- productora-web/README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 60,960

Tokens de salida (output): 24,722

Total tokens: 85,682

Horas-hombre estimadas: ~8.6 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: 4,166 input | 1,604 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: 13,541 input | 7,500 output | 5 llamadas

FRONTEND-DESARROLLO: 14,115 input | 15,433 output | 5 llamadas

QA-VALIDACION: 29,138 input | 185 output | 5 llamadas